

Mobile Robotics - Assignment 3

João Martins, José Carvalho, Lourenço Pinho and Maria Lopes

Once the low-level controller is complete, mobile robots can be deployed to perform various tasks that require planar movement. In such scenarios, a high-level controller is essential for planning and executing the robot's path. This assignment presents the formulation of multiple controllers designed for different types of common movements, such as following linear trajectories and curves. Additionally, an infrared sensor has been integrated into the system to enable a trajectory-following feature.

1 Kinodynamic Model

The robot used in this work will be a differential wheeled robot with two actuators in the form of motors. A typical example of such a robot can be visualized in Figure 1.

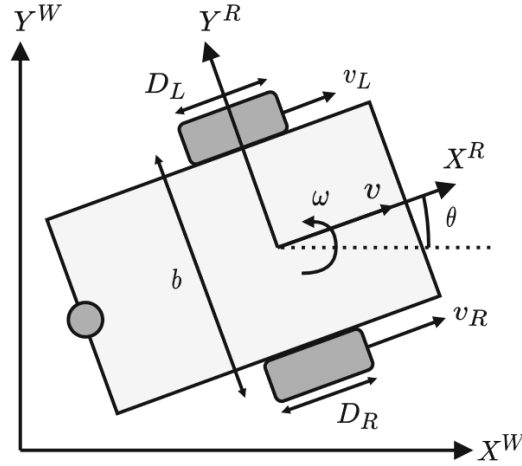


Figure 1: Example of Differential Wheeled Robot

The robot kinematics are nonlinear and have non-holonomic motion constraints, meaning that the 3 degrees of freedom (x, y, θ) are not decoupled from one another. The forward kinematics can be written as:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \underbrace{\begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix}}_J \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (1)$$

where v is the linear velocity on the robot's inertial frame, and ω is its angular velocity. These velocities can be related to the velocity of each wheel through the following equation:

$$\begin{bmatrix} v_L \\ v_R \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & -\frac{b}{2} \\ 1 & \frac{b}{2} \end{bmatrix}}_W \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (2)$$

in which b is the robot's width, v_L , and v_R are the velocities of the left and right wheels, respectively. The wheels' angular speeds are calculated using the relation with the wheel radius r :

$$\omega_w = \frac{v}{r} \quad (3)$$

and the subscript w will be used whenever referring to a generic wheel.

Considering that the motors operate outside their deadzone, the dynamical model can be approximated as a linear model with two poles. After performing system identification on the system, it was concluded that the pole associated with the electrical current is much smaller than the mechanical pole, and therefore, to simplify the model, it will be ignored. Thus, the motors will be described by a first-order model with the following transfer function on the Laplace domain:

$$G(s) = \frac{W(s)}{U(s)} = \frac{A}{\tau s + 1}, \quad (4)$$

where A is the transfer function gain, τ is the system's open-loop time constant, and $U(s)$ is the voltage signal or the control variable.

The goal of a generic closed-loop controller $H(s)$ will be to make it so that the system tracks the reference signal $R(s)$. In this section, we will simplify the controller synthesis by considering a generic controller that imposes the closed-loop dynamics described by:

$$\frac{1}{\tau' s + 1}, \quad (5)$$

Which is a unitary-gain first-order system with a time constant τ' . Considering that there is no wheel slippage or actuator saturation, robot velocities v, ω also follow the closed-loop dynamics of Equation 5, and thus the model can be extended as:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{v} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 0 & \sin \theta \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & \frac{-1}{\tau'} & 0 \\ 0 & 0 & 0 & 0 & \frac{-1}{\tau'} \end{bmatrix} \begin{bmatrix} x \\ y \\ \theta \\ v \\ \omega \end{bmatrix} + \begin{bmatrix} \frac{1}{\tau'} & 0 \\ 0 & \frac{1}{\tau'} \end{bmatrix} \begin{bmatrix} v_{ref} \\ \omega_{ref} \end{bmatrix}, \quad (6)$$

where v_{ref}, ω_{ref} are the reference velocity values that are given to the low-level controller. From these reference velocities, the individual wheel speeds can be computed by equation Equation 2. This model allows us to consider the wheel acceleration dynamics instead of just the kinematic constraints.

In this document, the design control architecture for the motion control of the robot will be explored. The control system was divided into two main blocks: a low-level and a high-level controller. The high-level block will be responsible for the path following control by receiving the robot's pose and orientation and computing reference velocities handled by the low-level controller. This framework can be visualized in Figure 2. It is important to note that the low-level controller and the state estimation were already addressed in previous reports. Therefore, the main focus will be on the high-level controller block, which will have multiple types of movements defined, all described in the sections below.

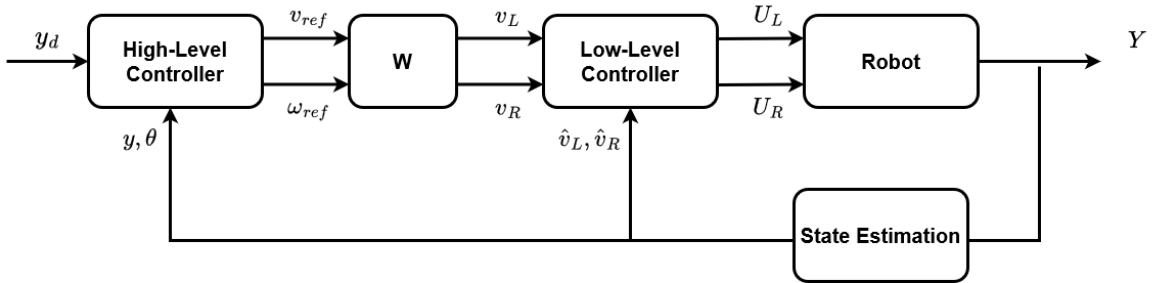


Figure 2: Proposed Controller Architecture.

2 SetTheta

Consider a robot described by the given forward kinematics. Let $\theta(t)$ be the robot's orientation and $\theta_d \in [-\frac{\pi}{2}, \frac{\pi}{2}]$ be a given desired orientation as represented in Figure 3. Design a feedback control law such that all the relevant closed-loop signals are bounded, and the regulation error $(\theta - \theta_d) = \tilde{\theta}$ converges to zero as $t \rightarrow \infty$.

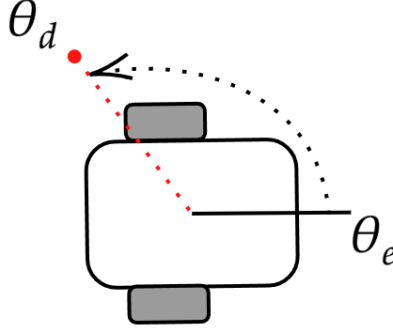


Figure 3: Set theta diagram

Since this is an orientation problem, the desired linear velocity, v_d , is defined as:

$$v_d = 0$$

Then the desired angular velocity ω_d is computed by the control law:

$$\omega_d = \begin{cases} \omega_{\text{nom}} \text{sgn}(\tilde{\theta}) & \text{if } |\tilde{\theta}| > \frac{\pi}{2} \\ \omega_{\text{nom}} \cdot \tilde{\theta} \cdot \frac{2}{\pi} & \text{if } |\tilde{\theta}| < \frac{\pi}{2} \end{cases}$$

Where the sgn operator enables the controller to move in the direction of the smallest rotation. Additionally, if $|\tilde{\theta}| < 0.01$:

$$\omega_d = 0$$

end_traj = true

3 GotoXY

Consider a robot described by the given forward kinematics. Let $p(t)$ be the robot's pose and $p_d \in \mathbb{R}^2$, defined by $p_d = (x_d, y_d)$, be a given desired point as illustrated in Figure 4. Design a feedback control law such that all the relevant closed-loop signals are bounded, and the regulation error $\|p(t) - p_d\| = \tilde{p}$ converges to zero as $t \rightarrow \infty$.

In this scenario, the following relations can be retrieved:

$$\begin{aligned} \theta_d &= \text{atan2}(y_d - y_e, x_d - x_e) \\ \tilde{\theta} &= \theta_d - \theta_e \\ \tilde{p} &= \|p(t) - p_d\| = \sqrt{(x_e - x_d)^2 + (y_e - y_d)^2} \end{aligned}$$

where $\tilde{p}$ is the pose error and $\tilde{\theta}$ is the orientation error. Then the desired angular velocity ω_d is computed as:

$$\omega_d = k_{\theta} \tilde{\theta}$$

The desired linear velocity v_d based on the following conditions:

$$v_d = \begin{cases} 0 & \text{if } |\tilde{\theta}| > \frac{\pi}{2} \\ v_{\text{nom}} \cdot \cos(\tilde{\theta}) \cdot \min(1, k_p \tilde{p}) & \text{if } |\tilde{\theta}| \leq \frac{\pi}{2} \end{cases}$$

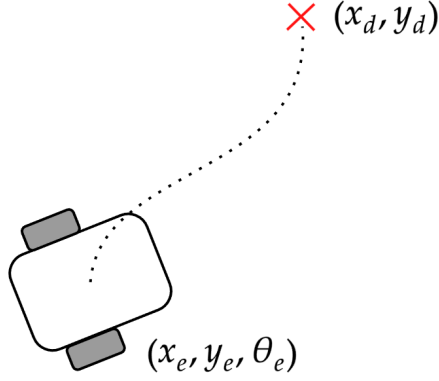


Figure 4: Goto XY diagram

Here, the $\cos(\tilde{\theta})$ term enables the robot to regulate its linear velocity if the angular error is large and the $\min(1, k_p \tilde{p})$ switches between applying the nominal velocity or implementing a proportional controller if the error is small enough. This effectively serves to give the robot a smoother deceleration. Additionally, if $|\tilde{p}| < 0.01$:

$$v_d = 0$$

$$\omega_d = 0$$

$$\text{end_traj} = \text{true}$$

With k_θ and k_p being the proportional gains defined as follows:

$$k_\theta = \frac{\omega_{\text{nom}}}{\frac{\pi}{2}}, k_p = \frac{1}{\lambda}, \lambda = 0.2$$

4 Follow Line

Consider the line segment that begins at the point (x_i, y_i) and ends at $(x_f, y_f) \in \mathbb{R}^2$. Then, let this path be parameterized by the function $\gamma(p(t)) : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, which, given the robot's pose, returns the point (x_l, y_l) in the path at the smallest euclidean distance, $\tilde{l}$, as represented in [Figure 5](#). Thus, the goal is to formulate a closed-loop control law so that all signals are bounded and the regulation error $\|p(t) - \gamma(p(t))\|$ tends to zero as $t \rightarrow \infty$.

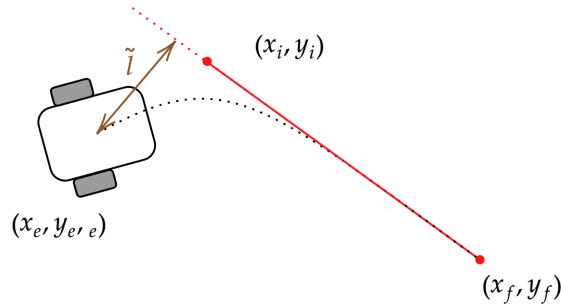


Figure 5: Follow line diagram

In this case, the relations retrieved are the following:

$$\theta_d = \text{atan2}(y_f - y_e, x_f - x_e)$$

$$\tilde{\theta} = \theta_d - \theta_e$$

$$\tilde{p} = \sqrt{(x_e - x_f)^2 + (y_e - y_f)^2}$$

The error to the line, $\tilde{l}$, can be computed with:

$$\begin{cases} u_x = \frac{x_f - x_i}{\|x_f - x_i\|_2} \\ u_y = \frac{y_f - y_i}{\|y_f - y_i\|_2} \end{cases}$$

$$\begin{cases} v_x = x_e - x_i \\ v_y = y_e - y_i \end{cases}$$

$$\tilde{l} = v_x \cdot u_y - v_y \cdot u_x$$

Let $\tilde{l}$ be the distance from the robot's pose $p(t)$ to the line segment, $\tilde{\theta} = (\theta_d - \theta)$ be the orientation error and k_y, k_θ be two proportional gains. Then, one can derive a control law for the orientation that will drive the robot to follow the line:

$$\omega_d = k_y \tilde{l} + k_\theta \tilde{\theta}$$

Finally, consider the pose error defined as $\tilde{p}(t) = \|p(t) - (x_f, y_f)\|$ the distance from the robot's pose to the end of the line, and k_p a proportional gain. Thus, we can determine the desired linear velocity v_d based on the following control law:

$$v_d = \begin{cases} 0 & \text{if } |\tilde{\theta}| > \frac{\pi}{2} \\ v_{\text{nom}} \cdot \cos(\tilde{\theta}) \cdot \min(1, k_p \tilde{p}) & \text{if } |\tilde{\theta}| \leq \frac{\pi}{2} \end{cases}$$

Additionally, if $|\tilde{p}| < 0.01$:

$$\begin{aligned} v_d &= 0 \\ \omega_d &= 0 \\ \text{end_traj} &= \text{true} \end{aligned}$$

4.1 Adaptative Gains

Consider the dynamic model presented by the forward kinematics given by equation [Equation 1](#). We are interested in following only straight lines, which, without loss of generality, can be described as the line $y = 0$ in an arbitrary reference frame. Thus, since the problem is path-following and not trajectory tracking, the state x can be ignored as we are only concerned with following the geometric path. Furthermore, due to $y = 0$, the orientation of the line is also $\theta = 0$, and as such, our model can be linearized via small angle approximation:

$$\begin{bmatrix} \dot{y} \\ \dot{\theta} \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & v(t) \\ 0 & 0 \end{bmatrix}}_A \begin{bmatrix} y \\ \theta \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \\ 1 \end{bmatrix}}_B \omega \quad (7)$$

where we consider that the velocity v is not a control variable and instead is a time-varying parameter of the model. Then, we can derive a full-state feedback controller:

$$\begin{bmatrix} \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & v(t) \\ 0 & 0 \end{bmatrix} \begin{bmatrix} y \\ \theta \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \end{bmatrix} [k_y \quad k_\theta] \begin{bmatrix} y \\ \theta \end{bmatrix} \quad (8)$$

which results in the closed-loop eigenvalues being computed as:

$$\begin{aligned} \begin{vmatrix} 0 & v(t) \\ -k_y & -k_\theta \end{vmatrix} &= s^2 + sk_\theta + k_y v(t) = 0 \Leftrightarrow \\ s &= \frac{-k_\theta \pm \sqrt{k_\theta^2 - 4k_y v(t)}}{2} \end{aligned} \quad (9)$$

and we will choose to have the imaginary part to be equal to zero, resulting in the expression for k_y being:

$$k_y = \frac{k_\theta^2}{4v(t)} \quad (10)$$

where the value of the real part is equal to $-\frac{k_\theta}{2}$, and as such we choose where to place the pole.

5 Follow Circle

Consider the circle that begins is centered at the point (c_x, c_y) and has a radius r . Then, let this path be parameterized by the function $\gamma(p(t)) : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, which given the robot's pose, returns the point (x_c, y_c) in the path at the smallest euclidean distance. Thus, the goal is to formulate a closed-loop control law so that all signals are bounded and the regulation error $\|p(t) - \gamma(p(t))\|$ tends to zero as $t \rightarrow \infty$. This problem is illustrated in [Figure 6](#).

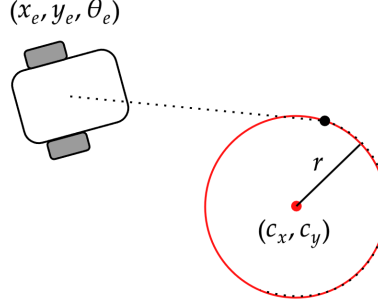


Figure 6: Follow circle diagram

Three distinct phases were used to solve the follow circle controller problem. The first one is a calculation of the best point to enter the circle; the second is the approximation to it using the GotoXY controller, where the robot converges to a point for better entering the circle without having to stop and rotate due to the differential wheel drive configuration. Then, upon being close enough, it enters the third and final phase of following the circle. This procedure is illustrated in [Figure 7](#), along with the equations used to calculate the references in each phase.

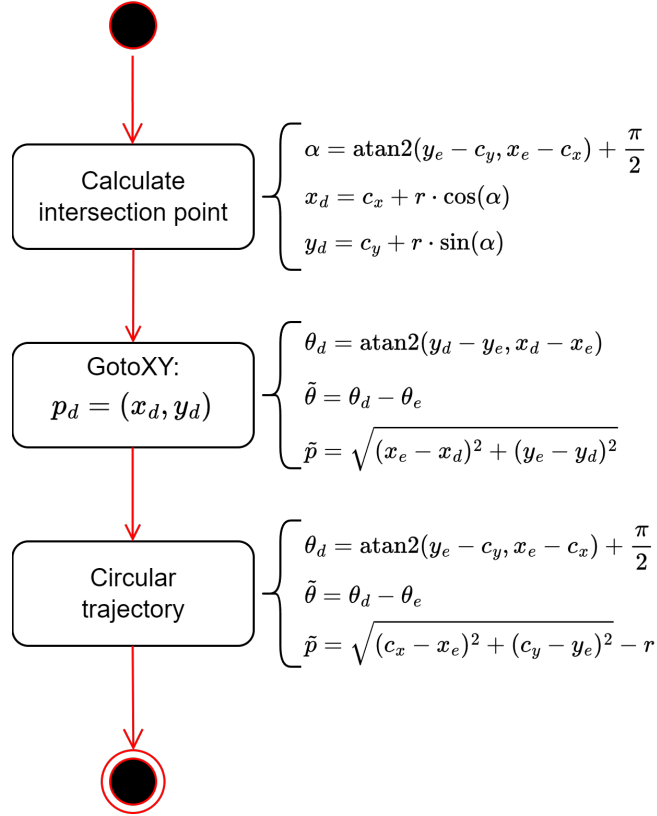


Figure 7: Follow circle state machine

Remember the proposed follow-line controller. If we consider that $\tilde{c} = \|p(t) - \gamma(p(t))\|$ is an error in the form of the distance from the robot's pose to the closest point in the circle and θ_d is the orientation of the line tangent to that point; then the proposed line controller will follow the circle path with the caveat of existence of a steady-state error due to $\dot{\theta}_d \neq 0$. This can be mitigated by adding a feedforward term to the controller:

$$\omega_d = k_c \tilde{c} + k_\theta \tilde{\theta} + \frac{v_d}{r}$$

Where v_d can be calculated the same way as in the line following case. For a continuous follow circle case, v_d is defined as v_{nom} .

6 Path Follower using IR Sensor

Consider the robot described by the given forward kinematics. Let it travel at a constant linear velocity v_d , and let $\tilde{\beta} \in \mathbb{R}$ be the displacement of the front robot in relation to the desired path. Design a feedback control law such that all relevant closed-loop signals are bounded and the regulation error $\tilde{\beta}$ converges to zero as $t \rightarrow \infty$.

Remember the proposed controller, GotoXY. For the case where IR sensors are used, a simpler version can be implemented:

$$\begin{aligned} v_d &= v_{nom} \\ w_d &= k_\beta \tilde{\beta} \end{aligned}$$

where k_β is a constant that relates the raw measurements from the IR sensors and the angular error, and $\tilde{\beta} = k \cdot \tilde{\theta}$ as the IR sensors do not provide the angle error directly. $\tilde{\beta}$ can be calculated from the IR raw values as follows:

$$\tilde{\beta} = \frac{4 \cdot IR_4 + 2 \cdot IR_3 - 2 \cdot IR_1 - 4 \cdot IR_0}{1024}$$

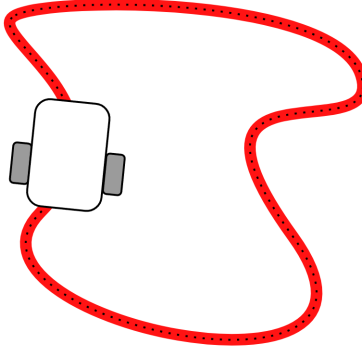


Figure 8: Path following diagram

7 Conclusions

The main conclusions of this work and the Mobile Robotics course from PDEEC are implementing a full framework from the low level to the higher level control of a differential kinematics robot. In previous reports, we explained the motor control and determined the intrinsic parameters. This one focused instead on the higher-level trajectories, from a simple setTheta controller to GotoXY, FollowLine, and FollowCircle.